

Supplementary Document SD5: Example of comparing strings using edit distances.

In order to illustrate the concept of how strings can be compared using edit distances, consider the following two strings

String 1: Breast Cancer

String 2: Brain Cancer

Let us now suppose, we can make the following operations: deletions, substitution, and insertion to transform string 1 to string 2. Then we can proceed to transform String 1 to String 2 in the following ways:

Table T1: step by step demonstration of comparing strings using edit distances.

Method 1	Method 1 operation	Method 2	Method 2 operation
Breast Cancer		Breast Cancer	
Bra s t Cancer	Delete 'e'	Bra a st Cancer	Substitute 'e' with 'a'
Bra s Cancer	Delete 't'	Bra i st Cancer	Substitute 'a' with 'i'
Bra Cancer	Delete 's'	Bra i nt Cancer	Substitute 's' with 'n'
Brai Cancer	Insert 'i'	Brain Cancer	Delete 't'
Brain Cancer	Insert 'n'		

We can observe from the above table T1, that the two methods transform string 1 to string 2, but method 2 employs fewer steps to achieve this objective. Furthermore, with the given set of operations the minimum number of steps required to transform string 1 to string 2 is four. This is achieved by method 2, thus the edit distance between string 1 and string 2 is four. The set of operations that were used to transform the strings and method with which we calculated the edit distance is also known as the Levenshtein distance.

Detailed description of edit distances used in CANTOS

Normalized Levenshtein distance: Levenshtein distance measures the minimum number of single-character edits (insertions, deletions, substitutions) needed to transform one string into another. Levenshtein distance between two strings can be normalized by dividing it by the length of the longer string thus the distance would range from [0,1] and allow comparison of multiple strings on the same scale. We defined the normalized Levenshtein distance between two strings S1 and S2 as follows:

$$Distance_{Normalized\ Levenshtein} = \frac{Distance_{Levenshtein}}{\max(|S1|, |S2|)}$$

In the above equation, $|S1|$ and $|S2|$ represent the respective lengths of strings S1 and S2 between which we are computing the normalized Levenshtein distance. We calculate Levenshtein distance using the stringdist library in the R programming language ^{111,112}. Following the calculation of the Levenshtein distance, we compute the normalizing factor (i.e. divide the Levenshtein by the longest string size) for distance between each pair of strings and normalize the Levenshtein distance.

Jarro-Winkler distance: The Jarro-Winkler distance is a normalized edit distance between two strings. It is a variant of the Jarro similarity measure which is defined as follows between two strings S1 and S2 respectively:

$$Sim_{jarro} = 0, \text{ if } m = 0$$

$$= \frac{l}{3} \left(\frac{m}{|S1|} + \frac{m}{|S2|} + \frac{m-t}{m} \right), \text{ otherwise}$$

Where $|S1|$ and $|S2|$ are lengths of the strings S1 and S2 respectively, m is the number of matching characters and t is the number of transpositions. It should be noted that when estimating 'm', each character in S1 compared to the characters in S2, and match is counted only when the characters are the same and if the characters are within a certain distance of each other typically defined as half the length of the longer string, rounded down, minus one, i.e. $\frac{\max(|S1|, |S2|)}{2} - 1$.

The Jarro-Winkler similarity measure builds on top of the Jarro similarity measure and introduces two more parameters for rewards and favorable scales the Jarro similarity score if the two strings share similar prefixes. The Jarro-Winkler similarity is defined as follows:

$$Sim_{jarro-winkler} = Sim_{jarro} + lp (1 - Sim_{jarro})$$

Where l is defined as the length of the common prefix at the start of the string (maximum of 4 characters), whereas p is a scaling factor that rewards the score for having common prefixes. Typically p is set to 0.1 and should not exceed 0.25 (or $\frac{1}{4}$ as the maximum length of prefix being considered is 4). With the above definition of Jarro-Winkler similarity, the Jarro-Winkler distance is defined as follows:

$$Distance_{jarro-winkler} = 1 - Sim_{jarro-winkler}$$

We calculate the Jarro-Winkler distance using the stringdist package in the R-programming language.

Cosine Distance: To define cosine distance, we first need to establish the concept of cosine similarity. For two non-zero vector vectors, cosine similarity is defined as the dot product of the two vectors divided by the product of their lengths. Cosine similarity ranges from $[-1,1]$, with -1 representing total opposition, 0 representing complete dissimilarity, and 1 representing full similarity between the vectors. Cosine similarity between two vectors A and B is defined as follows:

$$Sim_{cosine} = \cos(\theta) = \frac{A \cdot B}{|A| |B|}$$

However, to use cosine similarity in the context of strings, the vectors A and B represent the frequencies of unique words in strings S1 and S2. Since frequencies cannot be negative, the cosine similarity ranges between [0,1]. Thus, there is no need to normalize this metric, and cosine distance is defined simply as:

$$Distance_{cosine} = 1 - Sim_{cosine}$$

We calculate the cosine distance using the stringdist package in R.